

Can Retrieval-Augmented Few-Shot Prompting Match LoRA Fine-Tuning for Issue Report Classification? An Empirical Study

Anonymous Author(s)
Anonymous Affiliation(s)

Abstract—Classifying issue reports helps with software maintenance and triage. Fine-tuned large language models (LLMs) give the best reported results, but learning from new labels or adopting a newer model requires retraining. Retrieval-augmented few-shot prompting needs no training but, to our knowledge, has not been systematically evaluated for this task or compared with Low-Rank Adaptation (LoRA) fine-tuning. To fill this gap, we conduct an empirical study with four Qwen2.5-Instruct models and 6,600 issue reports from eleven projects. We first test whether a query issue’s nearest neighbors predict its label by taking a similarity-weighted vote over their labels ($\circ$), which reaches 0.595 macro F_1 . Our retrieval-augmented approach ($\circ$) therefore gives these neighbors to the LLM as few-shot examples. After observing that most often mislabels questions as bugs, we propose $\circ$, which drops bug-labeled examples for suspected questions. With the same 300 labeled issues per project, and at their best number of examples outperform LoRA fine-tuning of the same models by 0.021–0.060 macro F_1 at every size. When fine-tuning instead uses all eleven projects’ labeled issues, it leads by 0.028 on average and by 0.010, a difference that is not significant. With replacing invalid LLM outputs, matches fine-tuning (+0.001 on average). Needing only the target project’s labeled issues and 33% less peak GPU memory on average, retrieval-augmented few-shot prompting is a competitive, training-free alternative to LoRA fine-tuning. Classifying issue reports as bugs, feature requests, or questions helps developers triage them. Fine-tuned large language models (LLMs) give the best reported results, but fine-tuning needs a training run, reaches its best scores only with labeled issues pooled across projects, and must be repeated for new labels or a newer model. Retrieval-augmented few-shot prompting (RAG) instead shows the LLM the most similar labeled issues as examples and needs no training. To our knowledge, it has not been systematically evaluated for this task or compared with Low-Rank Adaptation (LoRA) fine-tuning. We conduct an empirical study with four Qwen2.5-Instruct models and 6,600 issue reports from eleven projects. RAG, like prior methods, most often labels questions as bugs, so a variant (filtered RAG) removes bug-labeled examples when the retrieved issues suggest a question. Using only the target project’s 300 labeled issues, filtered RAG comes within 1.0 point of fine-tuning on all 3,300 labeled issues of the eleven projects in macro F_1 , a difference that is not statistically significant, and needs no training and 33% less peak GPU memory on average. Without the filter, RAG trails fine-tuning by 2.8 points. Given the same 300 labeled issues, both RAG variants outperform fine-tuning at every model size. RAG’s inference is slower, and the two approaches make different errors: fine-tuning finds more bugs but over-predicts the bug label. With this filter, retrieval-augmented few-shot prompting preserves the macro F_1 of LoRA fine-tuning with $11\times$ less labeled data per project, no training, and a third less GPU memory.

Index Terms—Issue classification, Retrieval-augmented gen-

eration, Fine-tuning, Large language models, Mining software repositories

I. INTRODUCTION

Users and contributors of software projects report bugs, request features, and ask questions through issue trackers such as GitHub Issues [1]. Labeling each issue report with its type speeds up its resolution [2] and supports issue prioritization [3], issue assignment [4], and developer onboarding [5], [6]. Yet labels are rarely used [7] and often wrong [8], [9], and manually assigning them is laborious [10]. To address these problems, many studies propose methods for automated issue report classification ($\circ$) and typically evaluate them on the labels *bug*, *feature*, and *question* [7], [11]–[14]. Users and contributors of software projects report bugs, request features, and ask questions through issue trackers such as GitHub Issues [1]. Labeling each issue report with its type speeds up its resolution [2] and supports issue prioritization [3], issue assignment [4], and developer onboarding [5], [6]. Yet labels are rarely used [7] and often wrong [8], [9], and manually assigning them is laborious [10]. To address these problems, many studies propose methods for automated issue report classification (IRC) and typically evaluate them on the labels *bug*, *feature*, and *question* [7], [11]–[14].

Most methods learn from labeled issues. Early work trained classic machine-learning classifiers [7], [8], [15], later work fine-tuned pre-trained transformer models [11], [16]–[18], and recent work applies large language models (LLMs) [6], [13], [14]. Among LLM-based methods, fine-tuning on labeled issues gives the best reported results [6], [14], [19], with Low-Rank Adaptation (LoRA [20]) for open models. Fine-tuning, however, adds a training run that must be repeated to learn from newly labeled issues or to adopt a newer model [7], [10]. Most IRC methods learn from labeled issues. Early work trained classic machine-learning classifiers [7], [8], [15], later work fine-tuned pre-trained transformer models [11], [16]–[18], and recent work applies large language models (LLMs) [6], [13], [14]. Among LLM-based methods, fine-tuning on labeled issues gives the best reported results [6], [14], [19], with Low-Rank Adaptation (LoRA [20]) for open models. Fine-tuning has costs, however. Each model needs a training run, and even LoRA

training needs GPU memory beyond what inference needs. The best scores come from pooling the labeled issues of many projects into one training set [14]. The training must also be repeated to learn from newly labeled issues or to adopt a newer model [7], [10].

LLMs can also use labeled issues without training. Through in-context learning, a model infers a task from a few labeled examples in its prompt [21], and choosing the labeled examples most similar to the query issue can outperform choosing them at random [22], [23]. With such retrieval-augmented few-shot prompting, a newly labeled issue only needs to be indexed, and the LLM can be replaced without retraining. For future work, Heo and Lee [14] suggest few-shot prompting, and Aracena et al. [6] suggest retrieval augmentation to reduce the need for extensive fine-tuning and to help with ambiguous labels such as question. To our knowledge, however, retrieval-augmented few-shot prompting has not been systematically evaluated for or compared with LoRA fine-tuning. LLMs can also use labeled issues without training. Through in-context learning, a model infers a task from a few labeled examples in its prompt [21], and choosing the labeled examples most similar to the query issue can outperform choosing them at random [22]. With such retrieval-augmented few-shot prompting (RAG), a newly labeled issue only needs to be indexed, and the LLM can be replaced without retraining. For future work, Heo and Lee [14] suggest few-shot prompting, and Aracena et al. [6] suggest retrieval augmentation to reduce the need for extensive fine-tuning and to help with ambiguous labels such as question. To our knowledge, however, retrieval-augmented few-shot prompting has not been systematically evaluated for IRC or compared with LoRA fine-tuning.

This paper does both. In an empirical study, we use four Qwen2.5-Instruct models of 3B to 32B parameters [24] and Heo and Lee’s balanced benchmark of issue reports from eleven projects [14], each with 300 labeled and 300 test issues. Following Heo and Lee, the retrieval index and the fine-tuning data come either from the target project’s 300 labeled issues alone (the project-specific setting) or from all 3,300 labeled issues of the eleven projects (the project-agnostic setting). We conduct an empirical study that does both. We use four Qwen2.5-Instruct models of 3B to 32B parameters [24] and Heo and Lee’s balanced benchmark of issue reports from eleven projects [14], each with 300 labeled and 300 test issues. Following Heo and Lee, both the retrieval index and the fine-tuning data come either from the target project’s 300 labeled issues or from all 3,300 labeled issues of the eleven projects. We compare the approaches in macro F_1 , per-class errors, peak GPU memory, and time.

The retrieved examples are the labeled issues most similar to the query, so we first test how well their labels alone predict the query’s label. Voting-based TAGging (V) labels the query by a similarity-weighted vote over these neighbors, without an LLM. Using only the target project’s labeled issues, it reaches 0.595 macro F_1 , so the neighbors carry label information that an LLM could use. Retrieval-Augmented

Generative TAGging (R) therefore shows these neighbors to the LLM as examples. As with prior methods [13], [25], V’s most frequent error is labeling questions as bugs, and Balanced (B) targets this error by removing the bug-labeled examples for suspected questions. RAG shows the LLM the target project’s labeled issues most similar to the query (its neighbors) as examples. Two reference points separate what retrieval and the LLM each contribute. Zero-shot prompting is the LLM without examples. A similarity-weighted vote over the neighbors’ labels (the k NN voting baseline) is retrieval without the LLM and tests how much label information the neighbors carry. The vote alone reaches 59.5% macro F_1 , so the neighbors carry label information that the LLM can use. As with prior IRC methods [13], [25], RAG’s most frequent error is labeling questions as bugs. We hypothesize that bug-labeled examples retrieved for a question reinforce this error, and a variant (filtered RAG) removes them when the neighbors suggest a question, that is, when bug-labeled neighbors do not clearly outnumber question-labeled ones.

In this study, we answer four research questions. We answer three research questions.

How effectively can retrieval alone classify issue reports, and at what point does adding more retrieved neighbors stop helping? peaks at 0.595 macro F_1 with 15 neighbors from the target project and at 0.604 with 16 neighbors from all eleven projects. It is weakest on questions and labels 32% of them as bugs.

RQ1: *How well does retrieval-augmented few-shot prompting classify issue reports, and how many examples does it need?* With examples from the target project, RAG outperforms both zero-shot prompting and k NN voting at every number of examples, reaching 69.7–76.7% macro F_1 at its best k . Its most frequent error is still labeling questions as bugs (26–36% of questions).

How well does retrieval-augmented few-shot prompting classify issue reports, and how does performance vary with the number of in-context examples? shows the LLM k examples, for seven values of k from 0 (zero-shot prompting) up to 15, the bound set by V peaks. Every configuration with at least one example outperforms both zero-shot prompting and V’s best result. At each model’s best k , reaches 0.697–0.767 macro F_1 with the target project’s labeled issues. Retrieved examples lower the share of questions predicted as bugs from 46–59% at zero-shot to 26–36% at the best k , but question remains the weakest class.

RQ2: *Does filtering the retrieved examples reduce question-to-bug errors?* Filtered RAG lowers the share of questions labeled as bugs to 19–27% and raises macro F_1 over RAG by 1.5–2.4 points, at the cost of bug recall.

Can a balanced retrieval intervention reduce question-to-bug misclassification and improve retrieval-augmented few-shot performance? The models label many questions as bugs even without examples, so we hypothesize that bug-labeled neighbors retrieved for a question reinforce this error, therefore removes the

bug-labeled examples from the prompt, unless they outnumber the question-labeled ones by more than a small margin. It scores higher than at every $k \geq 6$ for all four models, and by 0.015–0.024 macro F_1 at each method’s best k (95% confidence intervals exclude zero). It also lowers the share of questions predicted as bugs to 19–27%, at the cost of bug recall.

RQ3: *Can RAG match LoRA fine-tuning with less labeled data and GPU memory?* Given the same 300 labeled issues, both RAG variants outperform fine-tuning at every model size. Fine-tuning pulls ahead only with all 3,300 labeled issues of the eleven projects. It then leads filtered RAG by 1.0 point on average, a difference that is not statistically significant, and leads RAG by 2.8 points. Both RAG variants need no training and 33% less peak GPU memory on average, but their inference is slower. Fine-tuning finds more bugs but over-predicts the bug label.

How do retrieval-based methods compare with LoRA fine-tuning in classification performance, peak GPU memory, and labeled-data needs? With the same 300 labeled issues of the target project, and at their best k score higher than LoRA fine-tuning at every model size, by 0.021–0.060 macro F_1 . When fine-tuning instead uses all 3,300 labeled issues of the eleven projects, trails it by 0.028 macro F_1 on average over the four model sizes, and trails it by 0.010, a difference that is not statistically significant. Once a fallback replaces invalid LLM outputs, matches fine-tuning (+0.001 macro F_1 on average). Beyond macro F_1 , fine-tuning finds more of the actual bugs but over-predicts the bug label, whereas makes fewer false bug predictions at most model sizes. The question-to-bug confusion persists under every method we test, including fine-tuning. Both retrieval-based methods need no training, 300 instead of 3,300 labeled issues, and 33% less peak GPU memory than fine-tuning on average. Overall, with a simple filter on its examples, retrieval-augmented few-shot prompting preserves the macro F_1 of LoRA fine-tuning with less labeled data per project, no training, and less GPU memory, and the two approaches make different errors. Our main contributions are the following:

Overall, retrieval-augmented few-shot prompting is competitive with LoRA fine-tuning while requiring no training, and the two approaches trade different strengths. Which one suits a project depends on its labeled data, its GPU budget, and whether a missed bug or a false bug label costs it more. Our main contributions are the following:

- a similarity-weighted k -nearest-neighbor baseline that measures how much label information the retrieved neighbors carry, bounds the number of in-context examples, and serves as a fallback for invalid LLM outputs. The first systematic evaluation of retrieval-augmented few-shot prompting for IRC, against LoRA fine-tuning of the same four models on the same labeled issues, covering macro F_1 , per-class errors, labeled data, peak GPU memory, and time, with a replication package.

- An evaluation of retrieval-augmented few-shot prompting (–) for across four model sizes and seven values of k . Evidence that, with a simple filter on its examples, it matches fine-tuning’s macro F_1 with $11\times$ less labeled data per project, no training, and a third less GPU memory, and that it beats fine-tuning when both use the same 300 labeled issues.
- a training-free retrieval intervention that reduces the question-to-bug misclassification known from prior work, at the cost of bug recall.
- A comparison of these methods with LoRA fine-tuning of the same models on the same labeled issues, in classification performance, peak GPU memory, and labeled-data needs, reported with paired bootstrap confidence intervals and per-class precision and recall. A training-free example filter that reduces the question-to-bug confusion known from prior IRC work, with an analysis of the error trade-offs between the approaches.

The rest of the paper is organized as follows. Section II describes our methods and the fine-tuning baseline; Section III the experimental setup, and Section IV the results. Section V analyzes failure cases and practical implications, Section VI reviews related work, Section VII discusses threats to validity, and Section VIII concludes and outlines future work. Section II describes the approaches, Section III the setup, Section IV the results, Section V the failure analysis and implications, Section VI related work, and Section VII the threats to validity; Section VIII concludes.

II. APPROACH

In this section, we describe the implementation details of our three proposed All retrieval-based IRC methods (classifiers share one retrieval step (Section II-B). The k NN voting baseline labels the query from its neighbors’ labels alone, RAG shows the neighbors to the LLM as labeled examples, – and –) as well as steps for faithfully reproducing the established LoRA fine-tuning baseline [6], [14]. Fig. 1 illustrates all four approaches to support the explanations. The remainder of the section is organized as follows: Section II-A explains the overall IRC objective. Section II-B details the indexing and retrieval pipeline shared by our three proposed retrieval-based methods. Section II-C presents –, our non-parametric method of classifying the query directly from its nearest neighbors. Section II-D describes –, our few-shot prompting technique that presents the nearest neighbors as classified examples in the prompt. Section II-E elaborates on –’s balanced retrieval intervention applied to –. Finally, Section II-F details the faithful reproduction of the fine-tuning baseline and filtered RAG removes bug-labeled examples for suspected questions. We compare the two RAG variants with LoRA fine-tuning of the same LLMs (Section II-F); Fig. 1 shows all four approaches.

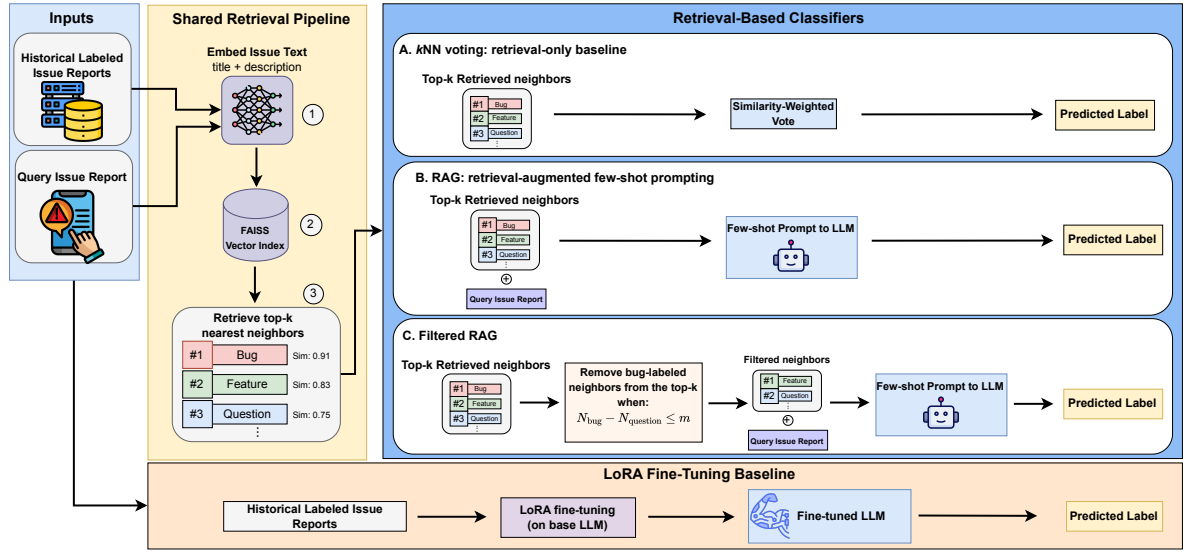


Fig. 1. Overview of the four IRC approaches evaluated in this paper: k NN voting baseline, retrieval-augmented few-shot prompting (RAG), filtered RAG, and the LoRA fine-tuning baseline.

A. Problem Formulation

Given a query issue report, we classify it into only one of the assign it exactly one of three labels: *bug*, *feature*, or *question*. Only We use only the issue's title and description text are used for classification, with no other signals from the issue tracker. The defined label space is adopted directly from label set follows prior IRC studies [6]–[8], [11], [19].

B. Indexing and Retrieval

To prepare the issues for retrieval based on textual similarity, we first embed their textual content (i.e., title + description). We do not embed the issue labels. Instead, we retain them as metadata associated with each embedded issue. To generate We embed each labeled issue's title and description and keep its label as metadata, outside the embedding. For the embeddings, we use the all-MiniLM-L6-v2 model from the Sentence-Transformers library [26]. We selected this model due to its lightweight architecture, fast performance, and effectiveness, which is small and fast and works well on issue report text [27]. The embeddings are then indexed using the FAISS vector database. We index the embeddings with the FAISS library [28], which provides efficient high-dimensional similarity retrieval. At the retrieval stage, At retrieval, the query issue is first embedded with the same model. Next, FAISS calculates the cosine similarity of indexed issue reports, and FAISS returns the top- k indexed issues by cosine similarity to the query, and returns the top- k nearest neighbors, ordered by descending (its nearest neighbors), in descending order of similarity.

C. k NN Voting Baseline

labels the query by a similarity-weighted vote over the labels of its top- k neighbors, following the distance-weighted k -nearest-neighbor rule [29], [30]. The k NN voting baseline labels the query by a similarity-weighted vote over the

labels of its top- k neighbors, following the distance-weighted k -nearest-neighbor rule [29], [30].¹ Because it uses no LLM, measures how much label information the neighbors carry, and its best k bounds the number of examples we show the LLM (Section IV-A). It also labels the issues whose LLM output is invalid when we evaluate complete pipelines (??). Because it uses no LLM, k NN voting measures how much label information the neighbors carry. Its peak sets the largest k we give the LLM (Section IV-A), and it labels invalid LLM outputs in the fallback (Section IV-C). Formally, for a query issue q with retrieved neighbors $\{(n_i, l_i, s_i)\}_{i=1}^k \{ (n_i, l_i, s_i) \}_{i=1}^k$, where n_i is the i -th nearest neighbor with label l_i and cosine similarity s_i to q , the predicted label $\hat{y}$ is given by:

$$\hat{y} = \arg \max_{c \in \mathcal{L}} \sum_{i=1}^k s_i \cdot \mathbf{1}[l_i = c], \quad (1)$$

where $\mathcal{L}$ is the label set, and $\mathbf{1}[l_i = c]$ is the indicator function that equals 1 when neighbor i has label c and 0 otherwise. For each label, k NN voting sums the cosine similarities of the neighbors that carry that label, and the arg max operator returns the label with the largest total sum. breaks ties by returning the Ties go to the tied label of the most similar neighbor in the tied classes.

D. Retrieval-Augmented Few-Shot Prompting (RAG)

applies retrieval-augmented few-shot prompting to . It shows the query's top- k neighbors (Section II-B) to the LLM as labeled examples. We use the neighbors, rather than randomly chosen labeled issues, because similar examples can outperform random ones [22], [23] and because a vote

¹Squared-similarity weighting and uniform an unweighted majority were also evaluated, but were outperformed by similarity-weighted voting vote scored lower.

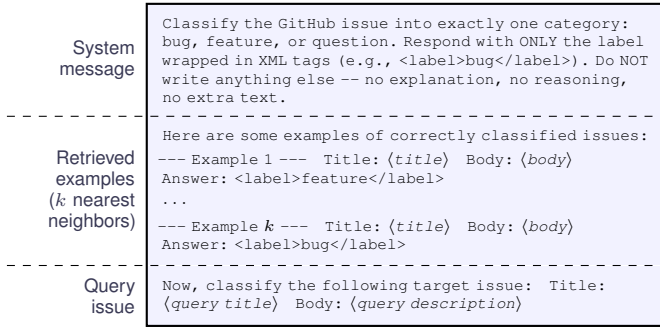


Fig. 2. The retrieval-augmented few-shot prompt structure used for RAG and filtered RAG.

over the neighbors' labels already reaches 0.595 macro F_1 (, Section IV-A). We RAG shows the query's top- k neighbors (Section II-B) to the LLM as labeled examples. We use the neighbors, rather than randomly chosen labeled issues, because similar examples can outperform random ones [22] and because the neighbors' labels alone reach 59.5% macro F_1 (Section IV-A). We create the few-shot prompt in three parts as shown in Fig. 2. First, the system message frames the classification task and the label space. Second, the top- k nearest neighbors are listed in retrieval order (descending similarity), each formatted as title + description + label. Finally, the query issue closes the prompt, since LLMs use information at the end of a long context better than in its middle [31]. With $k=0$, the prompt has no examples and becomes zero-shot prompting, as in prior work [13]. the query issue closes the prompt, since LLMs use information at the end of a long context better than in its middle [31]. With $k=0$, the prompt has no examples and RAG becomes zero-shot prompting, as in prior work [13].

We evaluate $k \in \{0, 1, 3, 6, 9, 12, 15\}$, up to 's peaks at $k=15-16$ (Section IV-A). We evaluate $k \in \{0, 1, 3, 6, 9, 12, 15\}$, up to k NN voting's peaks at $k=15-16$ (Section IV-A). The maximum prompt sequence length is set to 8,192 tokens.² Roughly 30% of the context quota this length is reserved for the query issue and 70% for the neighbors. If the neighbors exceed their overall quota, they are truncated proportionally based on their length (longer neighbors get more truncation). The query issue gets truncated if it exceeds its budget as well. Truncation happens on the right side of the description text, preserving examples. Examples that exceed their share are truncated in proportion to their length, and so is a query that exceeds its share. Truncation removes text from the end of a description and keeps all titles and labels (if any). The LLM is instructed to generate the predicted label wrapped wrap its label in XML tags (e.g., <label>bug</label>). The labels of the presented examples also follow the same format. We use a regular expression to extract the predicted label. Outputs

²Maximum prompt sequence length is selected empirically. In preliminary experiments, 8,192 tokens offered gave the best trade-off between macro F_1 and inference speed across our preliminary experiments.

that contain none of the three labels are marked as *invalid*. Finally, models are loaded, the format in which the examples show their labels. A regular expression extracts the label, and outputs that contain none of the three labels are *invalid*. We load the models with Unsloth³ for memory efficiency, and the temperature is set at to reduce memory use and set a low temperature (0.1 for deterministic classifications), so that outputs vary little between runs.

E. Balanced (+) Filtered RAG

Our empirical evaluations in ?? reveal that has the lowest class-specific F_1 on the question label, and it frequently misclassifies true-question queries as bugs, a known weakness [13], [25]. Furthermore, LLM zero-shot results also show that the models are already biased towards predicting the bug label for true-question queries even before the examples are presented. Based on these insights, RAG's most frequent error is labeling questions as bugs (Section IV-A), a known weakness of IRC methods [13], [25]. The models make this error even at zero-shot, so we hypothesize that presenting bug-labeled examples for true-question queries likely reinforces the question-to-bug misclassification trend. To address this, we propose a more balanced retrieval intervention, called balanced (+), that retrieved for a question reinforce it. Filtered RAG therefore removes the bug-labeled neighbors whenever examples from the prompt when the neighbors suggest that the query is a suspected question based on its nearest neighbors' label distribution. ?? shows that true-question queries retrieve almost balanced bug and question neighbors. Due to this balanced distribution, removes the bug-labeled examples from the prompt whenever question, that is, when $N_{\text{bug}} - N_{\text{question}} \leq m$, where N_{label} is the count number of neighbors with that label in the top among the top k and m is a small positive margin. The margin is empirically calibrated on a holdout validation set. The empirical motivations for and its margin calibration are fully detailed in ??. Other than the balanced retrieval intervention, inherits 's full setup and configurations. For example, with $k=12$ and $m=3$, a query whose neighbors are six bugs, four questions, and two features has $N_{\text{bug}} - N_{\text{question}} = 2$, so the six bug examples are removed and the prompt shows the remaining six.

We calibrated m before testing, on a validation split of the labeled issues. From each project's 300 labeled issues, 30 issues stratified by label serve as queries and the other 270 as the retrieval index, mirroring the project-specific setting (Section III). On this split, true questions retrieve nearly balanced bug and question neighbors, with on average 1.36 more question than bug neighbors over $k \in [1, 15]$. We swept $m \in \{1, \dots, 5\}$, where 5 is the 95th percentile of $N_{\text{bug}} - N_{\text{question}}$ for true questions, and chose $m=3$ as the best trade-off between firing for questions and for bugs. Averaged over k , it fires for 89.1% of true questions and 56.7% of true bugs. Otherwise, filtered RAG is identical to RAG.

³<https://unsloth.ai>

F. LoRA Fine-Tuning Baseline

Recent IRC approaches have achieved strong performance by supervised fine-tuning of LLMs [6], [14], [19], using Low-Rank Adaptation (LoRA) [20]. In Prior LLM-based IRC studies fine-tune open LLMs on labeled issue reports with LoRA [6], [14], [20]. Following these studies, the model is trained on labeled issue reports and predicts the labels for unseen test issues. Each training example is formatted as an instruction prompt that includes with the issue’s title and description as input, and its label as output. We adopt the instruction-style prompt template used in these prior LoRA fine-tuning studies [6], [14] verbatim, rather than designing our own, so that the baseline faithfully reproduces their established methodology. At inference, the trained fine-tuned model receives the test query issue in the same format, and predicts and completes the empty label field. Regular expressions are used to extract the prediction from the output, and outputs that contain none of the three labels are marked as A regular expression extracts the label, and outputs that contain none of the three labels are invalid. We use this established methodology as our fine-tuning baseline.

Training hyperparameters follow prior work [6], [14]: The where it reports them: LoRA rank and alpha are set to of 16, with a and a 2,048-token limit [6], and 3 epochs with a per-device batch size of 1 [14]. We use a learning rate of 2×10^{-4} and, the paged AdamW 8-bit optimizer. Training runs for 3 epochs with a per-device batch size of 1 and gradient accumulation steps of 16. The maximum prompt sequence length is set to [32], and 16 gradient-accumulation steps. The 2,048 tokens, since 048-token limit leaves 94.9% of the test issues in the dataset will not be truncated at this length. Issues that exceed this length get truncated from the right side untruncated, and longer issues lose the end of their description text to fit. Similar to and, the models are loaded. As for the RAG variants, we load the models with Unsloth and the temperature is set at set the temperature to 0.1.

III. EXPERIMENTAL SETUP

Dataset and Settings: We use the dataset introduced in Heo et al. [14] for our experiments. The dataset has a total of of Heo and Lee [14], which extends the five-project NLBSE’24 dataset [12] with six projects. It has 6,600 issue reports gathered from eleven active open-source projects: ansible, bitcoin, dart-lang, roslyn, react, flutter, kubernetes, TypeScript, vscode, opencv, and tensorflow. Each project has 600 issue reports with an equal distribution across the labels (*bug* divided equally among the three labels. Heo and Lee split each project’s issues in half, *feature*, and *question*). To support their LLM fine-tuning method, Heo et al. divided the dataset into 50/50 train and test splits, stratified by project and label. This results in label, into 300 train-labeled (training) and 300 test issue reports per project issues. We also adopt Heo et al.’s their two settings, which differ in where the labeled issues come from. In the project-specific (PS) and project-agnostic (PA) configurations to examine how data scope affects IRC

performance. In the PS setting, each project’s train and test data are used in isolation, resulting in eleven different train-test labeled issues are used alone, giving eleven training and test pairs. In the PA project-agnostic (PA) setting, the train splits of all projects are merged into a single-labeled issues of all eleven projects are pooled into one training set of 3,300 issue reports issues. In both configurations, and only use the training splits as their retrieval index to ensure fair comparison with the settings, the retrieval index holds exactly the issues that fine-tuning baselinetrains on.

Models: We use the Qwen2.5-Instruct model family [24] in our evaluations with four different parameter sizes: 3B, 7B, 14B, and 32B. Qwen2.5 is an open-source LLM with strong performance open LLM family with strong results on general-purpose benchmarks. Using the same model with different parameter sizes enables analysis of, and four sizes of one family let us study the effect of model scale in LLM-based IRC approaches size alone.

Evaluation Metrics: We report the macro-averaged F_1 over the three classes (*bug*, *feature*, *question*) as the headline metric and use it to select each method’s best configuration. We also report macro-averaged and per-class precision and recall. All metrics are calculated over the total 3,300-issue test set. We also report the rate of outputs that fail to parse to a valid label (*invalid rate*). Invalid outputs are counted as incorrect predictions. For statistical significance checks, we report paired bootstrap 95% confidence intervals on the macro F_1 differences between methods. We report the macro-averaged F_1 over the three labels as the headline metric, since it weighs the three labels equally. A method’s best k (its best configuration) is the one with the highest macro F_1 on the test set. We report scores in percent and differences between methods in percentage points (points), computed from unrounded scores. We also report macro-averaged and per-class precision and recall. We compute all metrics over the 3,300 test issues of all projects together. We also report the share of invalid outputs (*invalid rate*), which count as incorrect predictions. For significance, we report bootstrap 95% confidence intervals (CIs) on the macro F_1 differences between methods.⁴

Hardware Setup: Fine-tuning of Qwen-14B and Qwen-32B and the Qwen-32B runs at $k \geq 12$ used one NVIDIA L40 or L40S GPU (48 GB VRAM). All other experiments used one NVIDIA RTX 4090 GPU (24 GB VRAM). Fine-tuning of Qwen-14B and Qwen-32B and the Qwen-32B runs at $k > 12$ used one NVIDIA L40 or L40S GPU (48 GB VRAM). All other experiments used one NVIDIA RTX 4090 GPU (24 GB VRAM).

IV. EVALUATION

In this section, we present the results of our experiments and answer the four research questions posed in Section I

⁴We resample the 3,300 test issues 1,000 times and evaluate both methods on the same resample (paired bootstrap).

, one per subsection. Throughout this section, a method’s *best* k or configuration is the one with the highest macro F_1 . Fig. 3 reports macro F_1 across the k grid for the three retrieval-based methods, and Table I lists every method at its best configuration, including the per-class precision and recall that RQ2–RQ4 refer to. Fig. 3 shows macro F_1 across the k grid, and Table I lists every method at its best configuration with its per-class precision and recall. RQ1 evaluates RAG against its two reference points, RQ2 evaluates the example filter, and RQ3 compares both RAG variants with LoRA fine-tuning.

A. RQ1: Effectiveness of Retrieval-Only IRC Classification with Retrieved Examples

We evaluate in PS and PA at $k \in \{1, \dots, 20, 25, 30\}$ (Fig. 3a). Similarity-weighted voting achieves its highest observed macro F_1 of 0.595 at $k=15$ in PS and 0.604 at $k=16$ in PA. PA has slightly higher scores at every evaluated k . The small performance difference is consistent with the substantial overlap between retrieved neighbors: PS and PA share 84–90% of their top- k neighbors for the evaluated values between 3 and 30. Thus, even when the retrieval index includes all 3,300 training issues, most retrieved neighbors belong to the query’s project.

Question has the lowest class-specific F_1 at every evaluated k in both settings. At the peak configurations, 32% of question issues are predicted as bugs in both settings, compared with 18% predicted as features in PS and 17% in PA. Bug recall exceeds precision at every evaluated k , indicating that over-predicts this class. At the peak configurations, bug recall is approximately 0.73 and precision is 0.55–0.56 (Table I).

Retrieval-only voting achieves peak macro F_1 scores of 0.595 with project-specific data and 0.604 with pooled data. The small difference accompanies substantial overlap between retrieved neighbors. Question is the weakest class, with misclassifications more frequently assigned to bug than feature.

B. RQ2: Effectiveness of Retrieval-Augmented Few-Shot IRC

We evaluate at $k \in \{0, 1, 3, 6, 9, 12, 15\}$ in PS and PA, where $k=0$ is the zero-shot baseline [13]. ’s peaks at $k=15$ –16 (RQ1) set the upper end of this grid. Fig. 3b reports macro F_1 across the grid in both settings.

PS has slightly higher macro F_1 than PA at almost every (model, k) pair with $k \geq 1$, using 300 in-project retrieval examples rather than a pool of 3,300 examples. Zero-shot predictions are identical between settings because no retrieval is performed. The small PS–PA differences are consistent with the neighbor overlap observed in RQ1. We use PS for the remaining analysis.

All 48 nonzero- k configurations, spanning four model sizes and both data scopes, outperform the corresponding zero-shot baseline and the strongest configuration in macro F_1 , macro precision, and macro recall. Thus, LLM classification using retrieved examples outperforms similarity-weighted voting. Within the PS grid, the best k is 3, 6, 12, and 12 for Qwen-3B,

-7B, -14B, and -32B, respectively. These configurations achieve 0.697–0.767 macro F_1 , averaging a gain of 0.076 over zero-shot. Increasing k from 12 to 15 does not improve any model’s PS score.

Question remains the weakest class by F_1 at every evaluated k in both settings. It has the lowest recall but never the lowest precision. At zero-shot, 46–59% of true-question issues are predicted as bugs, compared with 5–16% predicted as features. These question-to-bug errors account for roughly 80% of false bug predictions, linking the question-recall deficit to low bug precision.

Retrieved examples reduce this confusion. At the best PS configurations, 26–36% of question issues are predicted as bugs and 9–15% as features. Averaged across the four models, the share of issues predicted as bug falls from 51% to 42%, bug precision increases from 0.57 to 0.66, and question recall from 0.35 to 0.51 relative to zero-shot (Table I). The increase in question-class F_1 accounts for 56–70% of the macro F_1 improvement across model sizes.

The zero-shot results show that question-to-bug confusion exists before any examples are presented. We hypothesize that bug-labeled examples retrieved for true-question queries reinforce this tendency. Based on this intuition, we propose a more balanced retrieval technique as an intervention that removes the bug-labeled neighbors whenever the query is a suspected question. We base the suspicion signal on the nearest neighbors’ label distribution and calibrate it without including the test set. From each We compare RAG with two reference points. Zero-shot prompting ($k=0$) is the LLM without examples and reaches 61.3–68.8% macro F_1 . The k NN voting baseline is retrieval without the LLM. We evaluate it at $k \in \{1, \dots, 20, 25, 30\}$ to find how many neighbors still add label information (Fig. 3a). Its macro F_1 rises slowly with k and peaks at 59.5% with the target project’s total 300 training issues, we assign 30 issues stratified by label as queries and 270 as the retrieval index, mirroring the PS setting. On this validation split, true-question queries retrieve almost balanced bug and question examples: averaged across $k \in [1, 15]$, the mean of $N_{\text{bug}} - N_{\text{question}}$ is -1.36. We therefore treat a query as a suspected question when $N_{\text{bug}} - N_{\text{question}} \leq m$, where N_{label} is the count of neighbors with that label in the issues ($k=15$) and at 60.4% with all projects’ issues ($k=16$); no larger k scores higher. These peaks set the top of RAG’s k grid.

The pooled index helps the vote only slightly. PA scores higher than PS at every k , but by little, because even the pooled index returns mostly neighbors from the query’s own project; PS and PA share 84–90% of their top- k and m is a small positive margin. Sweeping $m \in \{1, 2, 3, 4, 5\}$ ⁵ on the validation split and averaging across $k \in [1, 15]$, $m=3$ had a more favorable trade-off against the other values neighbors. The two settings therefore give the vote, and later the LLM,

⁵We cap the sweep at $m=5$, since it is the 95th percentile of $N_{\text{bug}} - N_{\text{question}}$ on true-question queries.

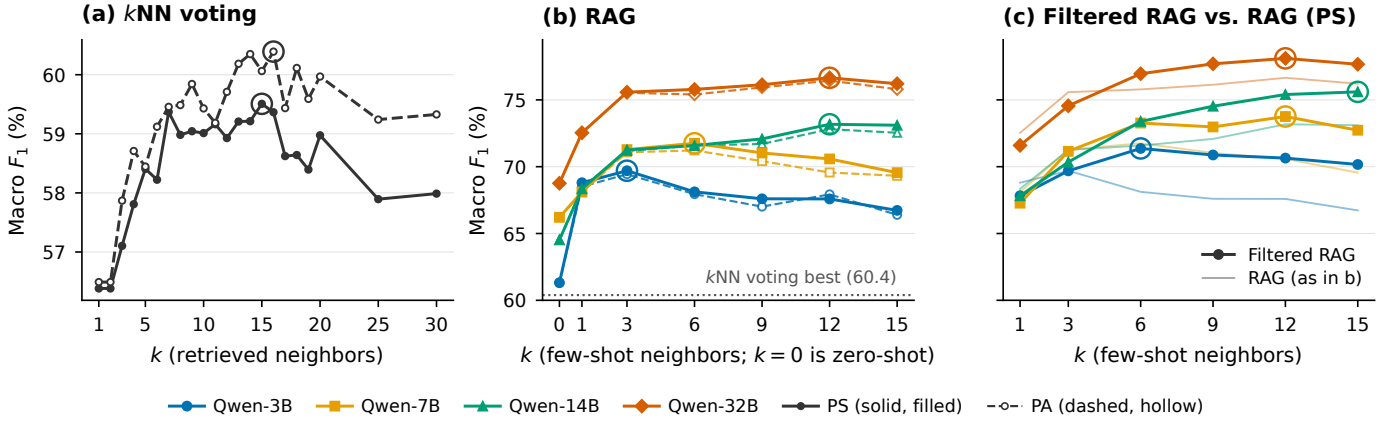


Fig. 3. Macro F_1 across the k -grid. (a) in PS (solid) and PA (dashed). (b) for the four Qwen sizes in PS (solid, filled) and PA (dashed, hollow). (c) (markers) against (b) and (c) share one y -axis and rings mark each method's best k . Macro F_1 (%) across the k -grid. (a) kNN voting in PS (solid) and PA (dashed). (b) RAG for the four Qwen sizes in PS (solid, filled) and PA (dashed, hollow). (c) Filtered RAG (markers) against RAG. (b) and (c) share one y -axis and rings mark each method's best k .

TABLE I
POOLED TEST-SET RESULTS RESULTS (%) OF EVERY METHOD AT ITS BEST CONFIGURATION ON ALL 3,300 TEST ISSUES.

Model	Method	k	Bug		Feature		Question		P_{mac}	R_{mac}
			P	R	P	R	P	R		
-	kNN voting (PS)	15	0.552	55.2	0.734	73.4	0.677	67.7	0.562	56.2
	kNN voting (PA)	16	0.556	55.6	0.731	73.1	0.689	68.9	0.579	57.9
Qwen-3B	Zero-shot	0	0.543	54.3	0.915	91.5	0.886	88.6	0.610	61.0
	RAG (PS)	3	0.638	63.8	0.803	80.3	0.794	79.4	0.813	81.3
	Filtered RAG (PS)	6	0.723	72.3	0.665	66.5	0.794	79.4	0.813	81.3
	Fine-Tune Fine-tuning (PS)	-	0.606	60.6	0.870	87.0	0.770	77.0	0.792	79.2
	Fine-Tune Fine-tuning (PA)	-	0.637	63.7	0.873	87.3	0.833	83.3	0.751	75.1
Qwen-7B	Zero-shot	0	0.575	57.5	0.881	88.1	0.792	79.2	0.789	78.9
	RAG (PS)	6	0.657	65.7	0.824	82.4	0.787	78.7	0.848	84.8
	Filtered RAG (PS)	12	0.713	71.3	0.774	77.4	0.761	76.1	0.855	85.5
	Fine-Tune Fine-tuning (PS)	-	0.653	65.3	0.635	63.5	0.759	75.9	0.769	76.9
	Fine-Tune Fine-tuning (PA)	-	0.672	67.2	0.878	87.8	0.825	82.5	0.800	80.0
Qwen-14B	Zero-shot	0	0.570	57.0	0.876	87.6	0.761	76.1	0.855	85.5
	RAG (PS)	12	0.651	65.1	0.844	84.4	0.839	83.9	0.836	83.6
	Filtered RAG (PS)	15	0.696	69.6	0.803	80.3	0.836	83.6	0.844	84.4
	Fine-Tune Fine-tuning (PS)	-	0.700	70.0	0.740	74.0	0.828	82.8	0.692	69.2
	Fine-Tune Fine-tuning (PA)	-	0.730	73.0	0.865	86.5	0.882	88.2	0.781	78.1
Qwen-32B	Zero-shot	0	0.611	61.1	0.861	86.1	0.766	76.6	0.869	86.9
	RAG (PS)	12	0.701	70.1	0.828	82.8	0.855	85.5	0.828	82.8
	Filtered RAG (PS)	12	0.748	74.8	0.804	80.4	0.847	84.7	0.833	83.3
	Fine-Tune Fine-tuning (PS)	-	0.744	74.4	0.700	70.0	0.814	81.4	0.825	82.5
	Fine-Tune Fine-tuning (PA)	-	0.689	68.9	0.929	92.9	0.832	83.2	0.876	87.6

PS: 300 labeled issues of the target project; PA: the 3,300 issues of all eleven projects. P/R : precision/recall; $P_{\text{mac}}/R_{\text{mac}}$: their macro averages. Invalid outputs count as incorrect. Bold: best macro F_1 per model size.

nearly the same neighbors. kNN voting also over-predicts bug and is weakest on questions, labeling 32% of them as bugs.

Without examples, firing for 89.1% of true-question and 56.7% of true-bug queries. We adopt $m=3$ and refer to the resulting method as **Balanced LLMs** over-predict bug. Zero-shot prompting assigns the bug label to about half of all issues, although a third are bugs, and labels 46–59% of questions as bugs but far fewer as features. These questions make up 76–84% of its false bug predictions, so low question

recall and low bug precision are two sides of the same error.

All 48 configurations with $k \geq 1$ (four sizes, six values of k , two settings) outperform both reference points in macro F_1 , macro precision, and macro recall (Fig. 3b). Even one example raises macro F_1 at every size. The best k grows with model size, from 3 at Qwen-3B and 6 at Qwen-7B to 12 at Qwen-14B and Qwen-32B, giving 69.7–76.7% macro F_1 . With examples, macro F_1 grows with every step in model size, whereas zero-shot prompting does not (Qwen-14B

scores below Qwen-7B). Beyond their best k , the two smaller models lose macro F_1 as examples are added, while the two larger ones stay nearly flat; no model improves from $k=12$ to $k=15$. We hypothesize that larger models make better use of long contexts with many examples. With largely the same neighbors, the two settings score nearly the same, and retrieving from the target project scores slightly higher at most sizes and values of k , so we use PS for RAG ~~⊖~~ and filtered RAG in the rest of the paper.

~~Retrieved examples improve macro F_1 , macro precision, and macro recall over zero-shot prompting and retrieval-only voting in every evaluated configuration with $k \geq 1$. The best PS configurations achieve 0.697–0.767 macro F_1 , and no model improves beyond $k=12$. Question recall remains the principal weakness: 26–36% of question issues are still classified as bugs, motivating the balanced retrieval intervention evaluated in RQ3. Most of RAG’s gain comes from questions. At the best k , the examples add 5.5–8.7 points to zero-shot prompting, and the rise in question F_1 accounts for 56–70% of this gain. The examples lower the share of questions labeled as bugs to 26–36%, and with it the share of issues labeled as bugs, which raises bug precision along with question recall. Question nevertheless keeps the lowest per-class F_1 at every k , and questions labeled as bugs remain the largest source of false bug predictions (74–79%). RQ2 targets this error.~~

~~B. RQ3: Effectiveness and Trade-offs of the Balanced Retrieval Method~~

RAG outperforms both zero-shot prompting and k NN voting at every number of examples, reaching 69.7–76.7% macro F_1 at its best k (3 to 12). The gain comes mostly from questions, but 26–36% of questions are still labeled as bugs.

~~We compare with under PS at the same $k \in \{1, 3, 6, 9, 12, 15\}$. At every tested $k \geq 6$, has higher macro F_1 for all four model sizes (Fig. 3c), by at least 0.012. At $k=1$ and $k=3$, its scores are lower by up to 0.010. With the fixed margin $m=3$, the condition $N_{\text{bug}} - N_{\text{question}} \leq 3$ holds for every query at these two small k values. Bug-example removal is therefore unconditional, leaving no demonstrations for 40% of queries at $k=1$ and 18% at $k=3$.~~

~~B. RQ2: Filtering the Retrieved Examples~~

~~Comparing each method’s best configuration, increases macro F_1 by 0.015–0.024 (Table I). The 95% CIs of these gains exclude zero for all four models: $[+0.002, +0.030]$, $[+0.007, +0.032]$, $[+0.014, +0.034]$, and $[+0.007, +0.022]$ ⁵.²s selected k increases for the three smaller models and remains 12 for Qwen-32B. When the trigger fires, removes all bug examples from the top-Fig. 3c compares filtered RAG with RAG at the same k . Filtered RAG scores higher at every $k \geq 6$ for all four sizes and lower at $k \leq 3$. The margin explains the losses at small k . With $m=3$ and at most three~~

neighbors, $N_{\text{bug}} - N_{\text{question}} \leq 3$ holds for every query, so the filter removes the bug-labeled examples unconditionally. It leaves 40% of the prompts at $k=1$ and 18% at $k=3$ without any example, which turns these queries into zero-shot prompts.

At each method’s best k , filtered RAG is 1.5–2.4 points higher than RAG, and the CIs of all four differences exclude zero. Its best k is larger at the three smaller sizes (6, 12, and 15 instead of 3, 6, and 12) and unchanged at Qwen-32B, and like RAG, it gains little or nothing beyond $k=12$. When the filter fires, it removes every bug-labeled example from the top k , so a higher larger initial k is needed to leave enough context for the LLM. examples in the prompt.

The class-level results expose the trade-off. Question F_1 increases by 0.030–0.068: recall rises by 0.065–0.153 while precision falls by 0.012–0.050. Bug precision increases by 0.045–0.086, but bug recall decreases by 0.025–0.138 at every model size. The largest recall loss occurs for Qwen-3B, from 0.803 to 0.665, the one model where the intervention over-corrects; its bug F_1 falls from 0.711 to 0.693. Bug F_1 improves for the three larger models. Feature F_1 changes by at most 0.012. The full precision and recall values appear in Table I.

At these configurations, question-to-bug error rates decrease from 26–36% with to 19–27% with . Question-to-feature error rates change by at most 1.4 percentage points. Averaged across $k \in [1, 15]$, the The gain comes from fewer question-to-bug rate drops from 31–40% to 24–36% errors. The share of questions labeled as bugs falls from 26–36% to 19–27% at the best k , and it also falls when averaged over all values of k , so the effect does not hinge on the choice of k . Question recall rises by 6.5–15.3 points, while question precision falls by 1.2–5.0 points. The precision loss comes mainly from bugs, because the filter also fires for many true bugs (Section II-E), and the share of bugs labeled as questions rises at every size. The improvement is therefore a targeted reduction of question-to-bug confusion, at the cost of bug recall. Invalid outputs remain counted as incorrect; their rates at these same configurations are reported in Table I and analyzed in RQ4.

On the bug label, the filter trades recall for precision. Bug precision rises and bug recall falls at every size. The loss is largest at Qwen-3B, where bug recall drops from 80.3% to 66.5% and bug F_1 drops by 1.8 points, the only size at which the filter over-corrects; at the three larger sizes, bug F_1 rises. At Qwen-3B, the share of bugs labeled as questions nearly doubles, far more than at the larger sizes, which suggests that the smallest model follows the remaining examples most closely. Feature F_1 changes by at most 1.1 points, so the filter leaves the feature label almost untouched (Table I gives all per-class values).

—outperforms— at every $k \geq 6$ for all four model sizes. The gain is a targeted reduction of question-to-bug misclassifications: question F_1 rises by 0.030–0.068 and bug precision by 0.045–0.086, at the cost of bug recall.

⁵1,000 paired-bootstrap resamples per model.

Bug F_1 improves for the three larger models and drops by 0.018 for Qwen-3B; feature F_1 changes by at most 0.012. Filtered RAG outperforms RAG at every $k > 6$, by 1.5–2.4 points at each method’s best k . It lowers the share of questions labeled as bugs to 19–27%, at the cost of bug recall.

C. *RQ4RQ3: Few-Shot Prompting vs. RAG versus LoRA Fine-Tuning for IRC*

Same labeled issues: We fine-tune the four models in both settings (Table I). Given the same labeled issues of the target project, both RAG variants outperform fine-tuning at every size, RAG by 2.1–4.0 points and filtered RAG by 3.8–6.0 points. Fine-tuned on these issues, the models still improve on zero-shot prompting at every size, but less than with retrieved examples. We hypothesize that 300 issues are too few for fine-tuning to learn the task well, whereas RAG needs only the few that are most similar to each query. The large gain that fine-tuning draws from more training issues supports this.

Pooling the labeled issues of all eleven projects raises fine-tuning’s macro F_1 by 3.1–8.4 points, consistent with Heo and Lee [14]. Pooling changes both the number of training issues and their project mix, and its effect differs by label: at Qwen-32B, for example, it raises bug recall but lowers question recall. Pooling does not help RAG, because the pooled index returns mostly the same neighbors (Section IV-A). We therefore compare each method in its better setting: the RAG variants with the target project’s issues (PS) and fine-tuning with the pooled issues (PA). The eleven PS indexes together hold the same 3,300 issues as the PA training set, so the settings differ in the labeled data available per target project, not in the total labeling effort.

We fine-tune all four models under both data scopes and compare them with and at their best k (Table I). When all methods use the same 300 labeled issues per project (PS), both retrieval-based approaches outperform fine-tuning at every model size: leads by 0.021–0.040 macro F_1 , and by 0.038–0.060. Fine-tuning benefits from pooling the 3,300 labeled issues across all eleven projects (PA), which changes both the number of training examples and their project composition. Pooling raises its macro F_1 by 0.031–0.084 relative to PS, consistent with prior findings [14], although the class-level effects vary: for Qwen-32B, it increases bug recall but lowers question recall (Table I).

The larger pool does not provide a corresponding benefit for retrieval (??). We therefore use PS for and and PA for fine-tuning in the remaining comparisons, giving each method its better-performing data scope. The eleven PS indexes collectively hold the same 3,300 training issues as the PA training set, so the two scopes differ in the data available per target project, not in the total annotation budget.

We report macro F_1 differences as the retrieval-based method minus fine-tuning, so negative values favor fine-tuning (Table II). Aggregated across the four model

TABLE II
MACRO- F_1 DIFFERENCE FROM PA FINE-TUNING IN PERCENTAGE POINTS, WITH 95% CIs, WITHOUT AND WITH THE k -NN VOTING FALLBACK (PAIRED BOOTSTRAP 95% CI).

Model	Fine-Tune RAG			Fine-Tune PA		
	ΔF_1	95% CI		ΔF_1	95% CI	
Qwen-3B	-0.011	-1.1	[-0.028, +0.007]	-2.8	+0.7	[-0.005, +0.5]
+ fallback	-0.015	-1.5	[-0.032, +0.002]	-3.2	+0.2	[-0.009, +0.9]
Qwen-7B	-0.044	-4.4	[-0.059, -0.029]	-5.9	-2.9	-0.024
+ fallback	-0.033	-3.3	[-0.047, -0.017]	-4.7	-1.7	-0.011
Qwen-14B	-0.054	-5.4	[-0.068, -0.039]	-6.8	-3.9	-0.029
+ fallback	-0.040	-4.0	[-0.054, -0.025]	-5.4	-2.5	-0.014
Qwen-32B	-0.005	-0.5	[-0.020, +0.009]	-2.0	+0.9	+0.010
+ fallback	+0.008	+0.8	[-0.008, +0.022]	-0.8	+2.2	+0.021
All sizes	-0.028	-2.8	[-0.039, -0.018]	-3.9	-1.8	-0.010
+ fallback	-0.020	-2.0	[-0.030, -0.010]	-3.0	-1.0	+0.001

ΔF_1 RAG and filtered RAG under PS at their best k , fine-tuning under PA; negative values favor fine-tuning. Each + row applies the fallback for: invalid outputs of both methods labeled by k -NN voting. Bold: CI excludes zero. All sizes: mean of the four per-model differences.

sizes, the difference is -0.028 for (paired bootstrap 95% CI [-0.039, -0.018]) and -0.010 for , whose CI [-0.020, +0.001] includes zero.⁵ trails fine-tuning at every model size, whereas shifts the difference in its favor at every size, to between -0.029 and +0.010. Fine-tuning is significantly ahead at Qwen-7B and Qwen-14B. At Qwen-3B and Qwen-32B, the confidence intervals include zero, and has the higher point estimates. Results also vary by project: matches or exceeds PA fine-tuning in 16 of the 44 (project, model) pairs, including all four sizes on openv and three of four on bitcoin (Fig. 4).

Pooled fine-tuning: Averaged over the four sizes, filtered RAG trails fine-tuning by 1.0 point, and the CI includes zero; RAG trails by 2.8 points, and the CI excludes zero (Table II). The filter thus turns a significant deficit into a difference that is not significant. Per size, RAG trails fine-tuning everywhere, significantly at Qwen-7B and Qwen-14B. Fine-tuning is significantly ahead of filtered RAG at Qwen-7B and Qwen-14B, while filtered RAG is ahead at Qwen-3B and Qwen-32B, but not significantly. Fine-tuning’s significant leads fall at the two sizes where pooling helps it most, by more than 8 points, against about 3 points at Qwen-3B and Qwen-32B.

Each method’s best k is chosen on the test set, which favors the RAG variants, since fine-tuning has no k to select. To check how much this matters, we fix $k=12$ for every size. Filtered RAG then trails fine-tuning by 1.2 points on average instead of 1.0, and it remains ahead at Qwen-32B.

Results also vary by project (Fig. 4). Filtered RAG matches or exceeds fine-tuning in 16 of the 44 (project, model) pairs, including all four sizes on openv and three on bitcoin, while fine-tuning leads at all four sizes on TypeScript and tensorflow. Differences on single projects reach about 10

⁵We use 1,000 paired bootstrap resamples per comparison.

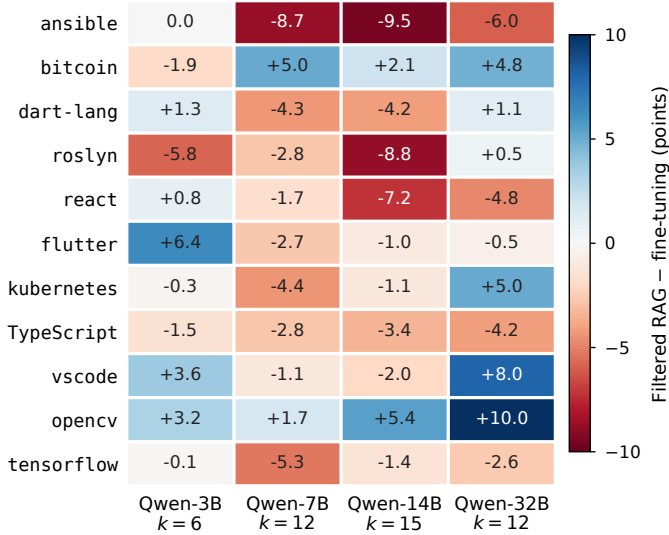


Fig. 4. Macro F_1 difference between (PS, best k) and PA LoRA fine-tuning on each project's 300 test issues, for the four model sizes. Blue favors $\text{filtered RAG (PS, best } k)$, red favors fine-tuning. Macro F_1 difference in points between filtered RAG (PS, best k) and PA LoRA fine-tuning on each project's 300 test issues, for the four model sizes. Blue favors filtered RAG, red favors fine-tuning.

points in either direction, so the parity in aggregate does not hold for every project.

Fine-tuning detects more actual bugs at every model size, while makes fewer false bug predictions at three of the four sizes. Like the zero-shot models (??), the PA fine-tuned models over-predict bug: they assign this label to 39–46% of test issues, although only 33% are bugs. Their bug recall is high (0.86–0.93), but precision is lower (0.64–0.73), and 23–36% of question issues are classified as bugs. predicts bug for 31–38% of issues, closer to the true share, and misclassifies 19–27% of questions as bugs. Its bug recall is lower (0.66–0.80), while its precision is generally higher (0.70–0.75). The one size at which does not make fewer false bug predictions is Qwen-14B, where fine-tuning's tendency to over-predict bug is less pronounced: it labels 39.5% of issues as bugs and misclassifies 23% of questions as bugs. A bug share close to the true proportion is not by itself evidence of quality; what matters is the trade-off between bug recall and bug precision, and which method is preferable depends on the relative cost of missed bugs and incorrect bug assignments in a given project.

At Qwen-7B and Qwen-14B, 's macro recall is lower than LoRA's by 0.035 and 0.043, respectively, while its macro precision is within 0.015 of LoRA's at every size. Invalid outputs contribute to this recall deficit: they count as false negatives for the true class without adding a prediction to any valid class's precision denominator. At the selected configurations, invalid rates are 0.2–4.6% for filtered RAG , 1.8–4.7% for fine-tuning , and at most 0.8% for LoRA. For filtered RAG at Qwen-7B and larger sizes, 6–8% of true bugs receive invalid outputs.

We therefore compare complete pipelines that assign a valid label to every issue by applying a fallback only when the LLM output is invalid. The fallback uses PS voting at fixed

TABLE III
COMPUTATIONAL COST OF k NN VOTING (PS, $k=15$), RAG, FILTERED RAG, AND LoRA FINE-TUNING AT THE CONFIGURATIONS OF TABLE II. RAM-MEMORY IS THE OBSERVED PEAK GPU MEMORY; TRAIN AND INFER ARE WALL-CLOCK RUNTIMES, AND TOTAL IS THEIR SUM.

Model	Method	RAM-Mem. (GB)	Train (h)	Infer (h)	Total (h)
–	k NN voting	0.2	–	<0.01	<0.01
Qwen-3B	RAG	2.9	–	0.18	0.18
	Filtered RAG	2.9	–	0.22	0.22
	Fine-Tune Fine-tuning	5.5	0.31	0.11	0.42
Qwen-7B	RAG	6.8	–	0.50	0.50
	Filtered RAG	6.8	–	0.60	0.60
	Fine-Tune Fine-tuning	9.9	0.48	0.10	0.58
Qwen-14B	RAG	12.6	–	1.37	1.37
	Filtered RAG	12.6	–	1.35	1.35
	Fine-Tune Fine-tuning	16.7	1.49	0.22	1.71
Qwen-32B	RAG	22.3	–	4.62	4.62
	Filtered RAG	22.3	–	4.15	4.15
	Fine-Tune Fine-tuning	31.5	2.28	0.44	2.71

$k=15$ for filtered RAG and PA voting at fixed $k=16$ for LoRA. It uses the original, unfiltered neighbors, independently of the number of demonstrations shown to the LLM. The retrieval-based pipelines can reuse their cached neighbor pool; adding this fallback to LoRA introduces a retrieval step. No additional LLM calls are required.

After applying the fallback to all three LLM approaches (Table II, + rows), 's differences from LoRA range from -0.014 to $+0.021$. Its deficits at Qwen-7B and Qwen-14B are approximately halved, to -0.011 and -0.014 , while its Qwen-32B advantage increases to $+0.021$. LoRA's scores change by at most 0.003. The aggregate difference between filtered RAG and fine-tuning becomes $+0.001$ (95% CI $[-0.009, +0.011]$), and per model the interval excludes zero only at Qwen-32B, in 's favor.

Our RAG-based methods require 47%, 31%, 25%, and 29% less peak GPU memory than LoRA fine-tuning for Qwen-3B, -7B, -14B, and -32B, respectively (33% on average), and remain competitive with it while using $11\times$ less labeled data per project (Table III). They have no training phase, but their longer prompts make each classification slower. Fine-tuned models label the 3,300 test issues in 0.10–0.44 h, whereas takes 0.22–4.15 h. Including training, takes less total time than fine-tuning at Qwen-3B and Qwen-14B, about the same at Qwen-7B, and more at Qwen-32B (4.15 h versus 2.71 h). Fine-tuning must, however, be repeated whenever the model is replaced or new labeled issues are added, whereas retrieval only needs to index the new issues. Without an LLM, labels the 3,300 test issues in under 4 s with 0.2 GB of GPU memory, at a macro F_1 of 0.595 (Section IV-A).

Labeled data, memory, and time: Filtered RAG uses $11\times$ less labeled data per target project than pooled fine-tuning and has no training phase. For each target project, fine-tuning learns from 3,300 labeled issues, whereas the RAG variants retrieve from that project's 300. Both RAG variants need 33%

less peak GPU memory on average (Table III), comparing their inference with fine-tuning’s training and inference of the same model. The saving ranges from a quarter at Qwen-14B to almost half at Qwen-3B, although the RAG prompts may be four times as long as fine-tuning’s (Section II-F).

The longer prompts make inference slower. Fine-tuned models label the 3,300 test issues in 0.10–0.44 h, and filtered RAG in 0.22–4.15 h. Including training, filtered RAG takes less total time than fine-tuning at Qwen-3B and Qwen-14B, about the same at Qwen-7B, and more at Qwen-32B. These totals include a single training run. Fine-tuning must be repeated whenever new labeled issues are to be learned or the model is replaced, whereas retrieval only indexes the new issues and uses any LLM as is. Conversely, the more issues are labeled between two training runs, the more fine-tuning’s faster inference pays off. Without an LLM, k NN voting labels all test issues in under 4 s with 0.2 GB of GPU memory.

Error profiles: Fine-tuning over-predicts bug, as zero-shot prompting does (Section IV-A), although its training data are balanced across labels. It labels 39–46% of the test issues as bugs, although 33% are bugs. It therefore finds more of the actual bugs than filtered RAG at every size, with bug recall of 86.5–92.9% against 66.5–80.4%, but it also makes more false bug predictions at three of the four sizes. The exception is Qwen-14B, where fine-tuning’s over-prediction is weakest. Per label, fine-tuning has the higher bug F_1 at every size, whereas filtered RAG has the higher question F_1 at Qwen-3B and Qwen-32B, the two sizes at which it leads in macro F_1 .

The question-to-bug confusion persists under both approaches, with 23–36% of questions labeled as bugs after fine-tuning and 19–27% after filtered RAG. Our failure analysis (Section V-A) shows that most of the sampled Qwen-32B errors of the three LLM approaches are issues filed under a bug-report template. Which error profile fits a project better depends on whether a missed bug or a false bug label costs it more.

Invalid outputs: Invalid outputs make up 1.8–4.7% of filtered RAG’s outputs and at most 0.8% of fine-tuning’s. In a sample, most of them continue the issue body or the model’s reasoning instead of giving a label (Section V-A). An invalid output counts as a missed issue of its true label but adds no prediction to any label’s precision, so it lowers macro recall but not macro precision. Filtered RAG’s macro precision is within 1.4 points of fine-tuning’s at every size, while its macro recall is lower at every size, most at Qwen-7B and Qwen-14B, where 6–8% of true bugs receive invalid outputs. We hypothesize that the long prompts, filled with issue text, lead the model to continue that text rather than answer.

To compare complete pipelines that give every issue a valid label, we let k NN voting label the invalid outputs of both methods. The RAG variants use PS voting at $k=15$ over their unfiltered neighbors, and fine-tuning uses PA voting at $k=16$. The fallback needs no extra LLM call; the RAG variants reuse their neighbors, and fine-tuning gains a retrieval step. With the fallback, filtered RAG is 0.1 point ahead of fine-tuning

on average, and the CI includes zero. Its deficits at Qwen-7B and Qwen-14B shrink to within 1.4 points and are no longer significant, and it becomes significantly ahead at Qwen-32B. Fine-tuning’s scores change by at most 0.3 points, since it has few invalid outputs, and RAG still trails it significantly on average.

Given the same 300 labeled issues per project, both retrieval-based methods outperform LoRA fine-tuning at every model size (– by 0.021–0.040 macro F_1 , – by 0.038–0.060). With $11\times$ more labeled data per project, fine-tuning leads by 0.028 in aggregate, but its 0.010 lead over is not statistically significant. Once a fallback handles invalid outputs, closes this gap (+0.001) and is significantly better at Qwen-32B. The methods trade different errors: fine-tuning detects more bugs but over-predicts the bug class, whereas makes fewer false bug predictions. Retrieval-augmented few-shot prompting also requires 33% less peak GPU memory than fine-tuning on average, but its inference is slower, and at Qwen-32B its total runtime is higher. With only the target project’s labeled issues, no training, and 33% less peak GPU memory on average, filtered RAG comes within 1.0 point of the macro F_1 of fine-tuning on the labeled issues of all eleven projects, a difference that is not statistically significant; RAG trails by 2.8 points. Given the same 300 labeled issues, both RAG variants outperform fine-tuning. Their inference is slower, and fine-tuning finds more bugs but over-predicts the bug label.

V. DISCUSSION

A. Failure Analysis

In this section, we conduct a manual qualitative analysis to better understand the misclassification patterns across the explored IRC methods. We inspect the misclassifications produced by–

We manually analyzed the errors of Qwen-32B under each method’s best configuration. From the full set of misclassifications produced by –, , and LoRA fine-tuning, we at each method’s best configuration. We randomly sampled 50 misclassified instances per method, yielding issues each from RAG, filtered RAG, and LoRA fine-tuning, 150 instances in total. Each method’s sample includes has 30 true-question, 10 true-bug, and 10 true-feature errors. The 30/10/10 split mimics the empirical class distribution of misclassifications pooled across –, which mirrors the label distribution of the errors pooled over the three methods. Each case was analyzed by two authors independently, with any discrepancies resolved through discussions. The observed failure categories and their frequencies are as follows: Two authors analyzed each case independently and resolved disagreements through discussion. We found three categories.

Issue reports whose content and ground-truth label contradict their chosen template and structure

Wrong template (116 of 150, 77%): The issue’s content and label contradict the template it was filed under. All of these errors are from questions and feature requests that the user filed under a bug-report template (*Steps to reproduce, Expected, Actual, System Information*). Consequently, the structure of the bug template misleads the LLM classifier towards predicting bug, whose structure leads the LLM to predict bug regardless of the issue’s content. For example, `vscode#193985`⁵ is an actual support question that opens with a literal `Type: Bug` because it uses VS Code’s bug template.

Issues that could have multiple defensible labels due to their combined intents. These errors mostly take place when

Hybrid intent (20 of 150, 13%): The issue combines intents and could defensibly carry more than one label, mostly because a fix or a feature request is layered into a question or bug report. For example, in `kubernetes#120364`⁶ the author both reports a concrete bug and, in the same body explicitly, requests a new feature: “Please, give us multiarchitecture image! and fix monoarchitecture images too!”

Issues whose

Label noise (14 of 150, 9%): The ground-truth label does not match their does not match the content. For example, `tensorflow#61710`⁷ is titled “Will there be an MLP model in the future version?” which requests a new API feature for building MLPs. Yet the dataset labels it as a bug, but is labeled as a bug.

Wrong templates affect all three approaches. We hypothesize that the headings of a bug template are a strong cue for the bug label, both for the retriever, which then returns bug-labeled neighbors, and for the LLM itself. The other two categories are harder for any text-based classifier. For the 34 errors with hybrid intent or label noise (23%), another label is defensible or the ground truth is wrong, so they limit the macro F_1 that any classifier can reach on this benchmark.

We also sampled 30 invalid predictions each from and invalid outputs each from RAG and filtered RAG. In 44 of the 60 cases (60 total) and observed three failure modes. In roughly 73% of cases, the model either continued to generate issue-body content continued the issue body or its own reasoning. Another 14 (23% were hallucinations), it produced degenerate text such as repeated digits and punctuation. The remaining 2 (3% are parsable labels that fall), it gave a label outside the three valid classes (e.g., such as documentation).

B. Key Insights and Practical Implications

Our results indicate that RAG-based approaches offer a practical solution for IRC. With 11× less labeled data per project and 33% less peak GPU memory on average, trails fine-tuning by 0.028 in aggregate macro F_1 , and the 0.010 gap to is not statistically significant. With a fallback for invalid

outputs, matches fine-tuning (??). RAG-based methods offer deployment advantages over

Labeled data and retraining: With filtered RAG, a project’s own labeled issues give, without training and with 33% less peak GPU memory, a macro F_1 within 1.0 point of fine-tuning as well: adding a on 11× as many issues pooled from eleven projects; the difference is not statistically significant, while RAG trails by 2.8 points (Section IV-C). For a project that cannot draw on other projects’ labeled issues, fine-tuning on its own issues scores 3.8–6.0 points below filtered RAG. The saving is per project, since the eleven indexes together hold as many labeled issues as the pooled training set. Building the index for a project needs only its labeled issues and one embedding pass, whereas pooled fine-tuning needs labeled issues collected from other projects. A new labeled issue report becomes an incremental then becomes an index update rather than a retraining eyele, run, so the examples can follow a project as its issues change, and the LLM can be replaced by a newer one without retraining.

Where fine-tuning keeps an edge: Fine-tuning keeps three advantages. It is significantly ahead at Qwen-7B and the model can be swapped in production with no project-specific retraining. For extremely resource-constrained scenarios, even PS is a defensible baseline: it reaches Qwen-14B, by 2.4 and 2.9 points, the sizes at which pooling the eleven projects’ issues helps it most (Section IV-C). Its inference is faster, most clearly at Qwen-32B, where filtered RAG takes longer in total even when fine-tuning’s training is included; this advantage grows with the number of issues labeled between two training runs. And it almost never produces invalid outputs, whereas the RAG variants need the k NN voting fallback to give every issue a valid label.

Choosing k : The best k differs by model size for both RAG variants, and beyond its best k , RAG loses macro $F_1 = 0.595$ ($k=15$), within 0.018 of Qwen-3B’s with the two smaller models (Sections IV-A and IV-B). Each added example also lengthens the prompt and so slows inference. A single $k=12$ for all four sizes keeps filtered RAG within 1.2 points of fine-tuning on average (Section IV-C), so the parity does not depend on tuning k per model.

Error profiles: The approaches differ less in macro F_1 than in which errors they make. Fine-tuning finds more bugs at every size, whereas filtered RAG raises fewer false bug alarms at three of the four sizes. At Qwen-32B, for example, fine-tuning reaches a bug F_1 of 79.1% against 77.5%, and filtered RAG a question F_1 of 72.9% against 66.9%. Macro F_1 weighs both errors equally, although a false bug label costs a triager’s time, whereas a missed bug can delay a fix. Neither approach removes the question-to-bug confusion.

Question-to-bug confusion: The confusion persists under every method we test, including fine-tuning, as in prior IRC work (Section VI). The filter reduces it without training but cannot remove it. The models favor the bug label even without examples (Section IV-A), and many of the remaining errors

⁵<https://github.com/microsoft/vscode/issues/193985>

⁶<https://github.com/kubernetes/kubernetes/issues/120364>

⁷<https://github.com/tensorflow/tensorflow/issues/61710>

are questions filed under a bug template (Section V-A), which read like bug reports to the retriever and the LLM alike.

Retrieval alone as a floor: Without an LLM, k NN voting reaches 59.5% macro F_1 , 1.8 points below Qwen-3B’s zero-shot 0.613 (Section IV-A, ??), at a fraction of the GPU cost (≈ 240 MB vs prompting, with 0.2 instead of 2.9 GB) of GPU memory. Neither the neighbors’ labels nor the LLM alone reaches the scores of RAG, which outperforms both at every size (Section IV-A); the gain comes from giving the LLM the labeled neighbors. The vote also labels fewer questions as bugs than zero-shot prompting does (Section IV-A), which suggests that the neighbors’ labels counter the LLM’s bias toward bug.

Classification and templates: Our failure analysis (Section V-A) shows that users frequently often file support questions and feature requests under the bug-report template, which can bias the classifier towards predicting leads the classifier to predict bug regardless of the content. This emphasizes the importance of refining the new Most of the sampled errors are of this kind, so correcting the template when an issue is filed could prevent many of them. This matters for the LLM-assisted triage pipelines now in place now offered on platforms such as GitHub [33] and GitLab [34]. In these pipelines, where the reporter describes the report issue to an LLM, which then suggests the correct that suggests a template and labels. Report validation is also important since mislabeling and duplication are common problems among contributors matters as well, since mislabeled and duplicate reports are common [8], [15]. Studies have already explored validation for Prior studies validate bug reports [35]–[38]. However, combining classification and validation in a cohesive workflow is underexplored, and a triage workflow could combine such validation with classification.

VI. RELATED WORK

Early IRC studies employed data mining and ML classifiers [7], [8], [10]. However, these methods yielded limited performance due to their shallow semantic understanding of issue report text [14]. To overcome this limitation, more recent work has adopted transformer models for their stronger contextual understanding

Supervised Issue Report Classification: Early IRC studies trained classic machine-learning classifiers on issue text [7], [8], [10]. Later work fine-tuned transformer encoders [11], [16], [17], [39], [40]. These studies typically fine-tune the models on historical issues in a supervised setting. For instance, Izadi et al. [11] and Trautsch et al. [16] fine-tuned RoBERTa [41] and seBERT [42] models, respectively, to classify issues into as bugs, features, or questions. Although transformer-based approaches achieve strong results, they rely on large amounts of training data, which limits their generalizability to settings where labeled data is scarce [11], [14], [17], [40]. These challenges have motivated the exploration of LLMs for IRC. Their extensive pretraining allows these models to be applied with

little or no task-specific training, reducing the dependence on the large labeled datasets that supervised classifiers require [1], [6], [13], [19], [43]. However, state-of-the-art LLM-based methods do not leverage this property. Instead, they fine-tune the LLM with Low-Rank Adaptation (and Siddiq and Santos trained a BERT classifier on more than 800,000 labeled issues [44]. Colavito et al. fine-tuned a sentence encoder with SetFit on about 200 labeled issues [43], [45]. All of these methods train a dedicated classifier on the labeled issues. This study instead holds the LLM fixed and varies only how the labeled issues are used, as examples in the prompt or as training data for LoRA, so its baseline is LoRA fine-tuning of the same models rather than a different classifier. Unlike SetFit’s encoder, ours is not trained and serves only to retrieve the neighbors, whose labels the LLM then sees as examples.

LLMs for Issue Report Classification: LLMs can classify issues without task-specific training [1], [13], [14], although their zero-shot performance varies across datasets [13]. Colavito et al. [1], [13] added one or two randomly chosen labeled examples per class to the prompt, and in their benchmark these examples did not improve over zero-shot prompting for most models [13]. The best reported results come from GPT models fine-tuned on labeled issues through OpenAI’s API [6], [14], [19]. Open models such as Llama-3.1-8B and DeepSeek-R1-Distill-Llama-8B were fine-tuned with LoRA [20] on labeled issue datasets to achieve their best reported performance [6], [14], [19]. Although and scored lower [6], [14]. Heo and Lee trained each classifier either on one project’s issues or on the issues pooled from eleven projects [14]. We adopt their benchmark and both settings. They average the scores of the eleven projects, whereas we score the test issues of all projects together, so our scores are not directly comparable with theirs. These studies report cost as training time, inference time, or API prices [6], [13], [14], not as GPU memory. For future work, Heo and Lee suggest few-shot prompting, and Aracena et al. suggest retrieving relevant past issues to reduce the need for extensive fine-tuning is effective, it reintroduces the dependency on labeled training data, remains computationally expensive, and must be repeated whenever the underlying model is swapped or as newly labeled issues accumulate in the project [7], [10].

Studies show that [6], [14]. This study evaluates both suggestions together. We show the LLM the labeled issues most similar to the query, up to 15 of them, and compare this with LoRA fine-tuning of the same four open models on the same labeled issues, in both of Heo and Lee’s settings, including peak GPU memory. RAG outperforms zero-shot prompting at every model size and every k (Section IV-A). Pooling the eleven projects’ issues helps fine-tuning, as Heo and Lee found, but not RAG, because the pooled index returns mostly neighbors from the query’s own project (Sections IV-A and IV-C). To our knowledge, retrieval-augmented few-shot prompting can

improve LLM performance in specialized tasks due to in-context learning [21], [46]–[49]. Few-shot prompting has also been applied in IRC. Colavito has not been systematically evaluated for IRC or compared with LoRA fine-tuning before.

Retrieved In-Context Examples: In-context learning lets an LLM infer a task from labeled examples in its prompt [21]. Choosing the examples most similar to the query outperforms random choice for GPT-3 on NLP benchmarks [22], and Milios et al. retrieve examples with a frozen sentence encoder to classify intents and emotions with up to 150 labels [50]. Yu et al. [1], [13] randomly selected one or two labeled examples per class from their training data and presented them in a few-shot prompt. In their benchmark, these examples did not improve over zero-shot prompting for most models [13]. Few-shot performance is sensitive to the quality of in-context examples. Research shows that using retrieval techniques to select the most semantically relevant, by contrast, train both the retriever and a RoBERTa classifier [23]. For masked language models of under one billion parameters, Logan et al. found fine-tuning more accurate than in-context examples can outperform random selection [22], [23]. Since retrieval-augmented few-shot prompting can enhance performance without the overhead of training, it can be a promising alternative to the LoRA learning and more robust to the prompt wording [49]. In software engineering, LLM-Cure assigns app reviews to features with five fixed examples in its prompt [46]. Closest to our work, Dinc and Tüzün show LLMs the five most similar labeled reports, retrieved with the same sentence encoder and FAISS library as ours, to decide whether Firefox bug reports are valid. In their study, fine-tuned encoders score higher than the prompted LLMs, and the authors name a sweep over k and filtering the neighbors as future work [38]. This study takes both steps for three-label IRC across eleven projects. It sweeps k up to where a vote over the neighbors' labels peaks and filters the examples (Section IV-B). It also separates what retrieval and the LLM contribute with the k NN voting and zero-shot references (Section IV-A) and compares with LoRA fine-tuning IRC baseline. Recent IRC studies also point in this direction in their future work. Heo and Lee [14] suggest few-shot prompting to improve accuracy across diverse projects, and Aracena et al. [6] propose retrieval augmentation, in which relevant past issues are retrieved and added to the LLM's prompt, to reduce the need for extensive fine-tuning. To the best of our knowledge, however, this method has not been systematically and empirically evaluated for IRC, of the same 3B to 32B instruction-tuned LLMs. In our study, RAG trails fine-tuning on the pooled issues by 2.8 points; with the example filter, the gap shrinks to 1.0 point and is not statistically significant (Section IV-C).

Label Bias in the Examples: Ma et al. observe that similarity-based selection tends to pick examples whose labels match the query's, and that LLMs then lean toward the labels that dominate the context [48]. Milios et al. report

that the neighbors retrieved for the neutral emotion rarely carry that label, and suspect that retrieval limits performance on it [50]. We see a similar pattern for questions. On our validation split, true questions retrieve nearly as many bug-labeled as question-labeled neighbors (Section II-E), and the models label many questions as bugs even without examples (Section IV-A). We hypothesize that the bug-labeled examples reinforce this error. Ma et al. reduce the bias by searching, with extra LLM calls, for one low-bias set of examples, scored by the entropy of the LLM's label distribution on a content-free input [48]. Dinc and Tüzün add a judge LLM that sees the votes of several classifiers [38]. Filtered RAG instead keeps per-query retrieval and applies a count rule to the neighbors' labels, removing the bug-labeled examples for suspected questions without an extra LLM call or access to output probabilities. It raises macro F_1 over RAG by 1.5–2.4 points at the cost of bug recall (Section IV-B).

In studies that include both bug and question labels, question is often the hardest class to identify, and misclassified questions are most often labeled as bugs. For example, almost all methods in the NLBSE'24 tool competition struggled with question [12], and on the same dataset, the two best-performing zero-shot LLMs labeled 46–54% of questions as bugs [13]. Fine-tuned decoder LLMs also score lowest on question in aggregate [6], [14]. Our results show the same pattern. Every method we evaluate misclassifies questions as bugs, with predicting bug for 32% of questions, zero-shot prompting for 46–59%, and PA fine-tuning for 23–36% (Section IV-A and ???). As a remedy, Aracena et al. [6] suggested retrieval augmentation, particularly for ambiguous labels such as question. Our RAG-based methods show that it helps but does not completely eliminate the problem. Without any training, retrieved examples lower the share of questions predicted as bugs from 46–59% to 26–36%, and lowers it further to 19–27%.

Question-to-Bug Misclassification: In IRC studies that include both bug and question labels, question is often the hardest class to identify, and misclassified questions are most often labeled as bugs. For example, almost all methods in the NLBSE'24 tool competition struggled with question [12], and on the same dataset, the two best-performing zero-shot LLMs labeled 46–54% of questions as bugs [13]. Fine-tuned LLMs also score lowest on question in aggregate [6], [14]. Our results show the same pattern: every method we evaluate, fine-tuning included, labels many questions as bugs (Sections IV-A and IV-C). As remedies, Aracena et al. [6] suggest retrieval augmentation, particularly for ambiguous labels such as question, and a first classification stage that separates questions from bug and feature reports. In our study, retrieved examples lower the share of questions labeled as bugs from 46–59% under zero-shot prompting to 26–36%, and filtered RAG lowers it further to 19–27% without an extra classification stage, but neither removes the confusion.

Prior work offers several explanations for why questions are misclassified as bugs. A common reason is label imbalance,

since question is often the smallest class in the training data [11], [39]. Another explanation is that some questions read like bug reports because they describe unexpected behavior, contain error messages [25], or lack explicit interrogative markers [6]. Our failure analysis (Section V-A) shows the same effect for questions filed under a bug-report template, which led the model to predict bug regardless of their content. Finally, a label can depend on information outside the report, such as a maintainer’s later diagnosis [25]. Likewise, the label-noise cases in our failure analysis have labels that do not match their content and cannot be inferred from the report text alone. Prior work offers several explanations for this confusion. Question is often the smallest class in the training data [11], [39]. Our models favor the bug label even before they see any labeled issue, labeling many questions as bugs at zero-shot (Section IV-A); we hypothesize that they learned in pretraining that bug reports outnumber questions. Some questions read like bug reports because they describe unexpected behavior, contain error messages [25], or lack explicit interrogative markers [6]. Our failure analysis (Section V-A) finds a related cause: questions filed under a bug-report template lead the model to predict bug regardless of their content. Finally, a label can depend on information outside the report, such as a maintainer’s later diagnosis [25]. Likewise, the label-noise cases in our failure analysis have labels that do not match their content and cannot be inferred from the report text.

VII. THREATS TO VALIDITY

Internal Validity: ~~’s margin uses the same empirically derived fixed value across~~ Each method’s best k is the one with the highest macro F_1 on the test set, which favors the RAG variants. With a single $k=12$ for every model size, filtered RAG trails fine-tuning by 1.2 points on average instead of 1.0 (Section IV-C). The filter’s margin was calibrated on a validation split of the labeled issues, not on the test set, and is the same for all models and projects with no per-project or per-model tuning, and is not framed as a universally optimal constant. Furthermore, the choice of hyperparameters and prompt format can also pose an internal threat to validity as different hyperparameters and prompts can change (Section II-E). Prompts and hyperparameters affect fine-tuning performance [6], [14] results [6], [14]. Ours follow prior work where it reports them (Section II-F), and other choices could change the results of either approach. The two approaches also have different length limits, but fine-tuning’s 2,048 tokens leave 94.9% of the test issues untruncated. Invalid outputs count as errors, which penalizes the RAG variants most; the fallback analysis shows their effect (Section IV-C). The issues are public and may appear in the LLMs’ pretraining data, which would affect zero-shot prompting and both approaches, since all use the same models.

Construct Validity: Macro F_1 on a balanced three-label test set weighs the labels equally, and we report per-class

precision and recall so that trade-offs between labels stay visible (Table I). Some ground-truth labels are wrong, accounting for 9% of the sampled errors (Section V-A). Since all methods are scored against the same labels, this noise lowers every method’s scores. LLM outputs can vary between runs, so we ran the experiments three times and report the average. The failure analysis covers 150 sampled errors of Qwen-32B, so its shares are estimates and may differ at other sizes. Times depend on the GPU, and at Qwen-14B, fine-tuning and the RAG variants ran on different GPUs (Section III).

Conclusion Validity: We test differences between methods with paired bootstrap CIs that resample the test issues (Section III). The per-size comparisons are not corrected for multiple comparisons, so a single per-size result should be read with care; the headline rests on the average over the four sizes. The CIs resample issues, not projects, so they do not capture the variation across projects, which reaches about 10 points on single projects (Fig. 4).

External Validity: Issue-report text can vary substantially. We study one model family (Qwen2.5-Instruct, four sizes), one embedding model, and one benchmark of eleven GitHub projects with balanced labels. Results may differ with imbalanced labels, other issue trackers, or other model families. The $11\times$ ratio follows from pooling eleven projects with 300 labeled issues each, and other numbers of projects or of labeled issues per project may change the gap between the approaches. Issue text also varies in writing style, vocabulary, and structure across software projects [51], which may threaten the generalizability of our findings. Another threat to external validity is generalizability across settings. We note that many configurations, including LLM architecture, prompt structure, embedding model, and retrieval technique, can each lead to different results. Assessing how far our findings generalize to other settings is itself an important question, but one that falls outside this paper’s scope and warrants its own dedicated studies. The inherent randomness of LLM outputs can pose a potential threat to construct validity. The reported performance metrics can vary between different runs due to LLM randomness. To mitigate this variance, we ran the experiments three times and report the average, and our results vary by project as well (Fig. 4).

VIII. CONCLUSION AND FUTURE WORK

In this study, we investigated retrieval-augmented few-shot prompting as a training-free alternative to LoRA fine-tuning for issue-report classification ($\circ$). We conducted an empirical study with four Qwen2.5-Instruct models and 6,600 issue reports from eleven projects, where the retrieval index and the fine-tuning data come either from the target project’s 300 labeled issues or from all 3,300 labeled issues of the eleven projects. We first tested how well the labels of a query issue’s most similar neighbors predict its label, using a similarity-weighted vote over them ($\circ$). Since these neighbors carry label information, our retrieval-augmented approach ($\circ$)

gives them to the LLM as few-shot examples. Because most often labels questions as bugs, its balanced variant ($\cdot$) removes the bug-labeled examples for suspected questions.

Retrieved examples improve the LLM over both zero-shot prompting and $\cdot$, and improves on $\cdot$ by reducing question-to-bug errors, at the cost of bug recall. With the same labeled issues of the target project, and at their best number of examples outperform LoRA fine-tuning at every model size. When fine-tuning instead uses all 3,300 labeled issues of the eleven projects, it outperforms $\cdot$, while its lead over $\cdot$ is not statistically significant. With replacing invalid LLM outputs, $\cdot$ matches fine-tuning ($+0.001$ macro F_1 on average). Fine-tuning finds more of the actual bugs but over-predicts the bug label, whereas $\cdot$ makes fewer false bug predictions at most model sizes, and the question-to-bug confusion persists under every method we test. Needing only the target project's labeled issues and 33% less peak GPU memory on average, retrieval-augmented few-shot prompting is a competitive, training-free alternative to LoRA fine-tuning.

The two approaches trade different strengths, and which one suits a project depends on its labeled data, its GPU budget, and whether a missed bug or a false bug label costs it more. With retrieval, a newly labeled issue only needs to be indexed, and the LLM can be replaced without retraining. As this work constitutes a case study on the [We studied retrieval-augmented few-shot prompting \(RAG\) as a training-free alternative to LoRA fine-tuning for issue report classification \(IRC\). We conducted an empirical study with four Qwen2.5-Instruct family-Instruct models of 3B to 32B parameters and 6,600 issue reports from eleven projects, taking the labeled issues either from the target project alone or from all eleven projects. Zero-shot prompting and a similarity-weighted vote over the neighbors' labels \(\$k\$ NN voting\) served as reference points for what the LLM and retrieval each contribute. Because RAG, like prior methods, most often labels questions as bugs, we also evaluated a variant \(filtered RAG\) that removes bug-labeled examples when the neighbors suggest a question.](#)

With this filter, retrieval-augmented few-shot prompting matches the macro F_1 of LoRA fine-tuning while using $11\times$ less labeled data per project, no training, and 33% less peak GPU memory on average. Using only the target project's labeled issues, filtered RAG trails fine-tuning on the labeled issues of all eleven projects by 1.0 point, a difference that is not statistically significant, while RAG trails by 2.8 points. Per size, fine-tuning is significantly ahead at Qwen-7B and Qwen-14B, and filtered RAG is ahead at the other two sizes, but not significantly. Given the same 300 labeled issues, both RAG variants outperform fine-tuning at every model size. Two earlier results lead to this one. Retrieved examples improve the LLM over both reference points at every number of examples, mostly on questions, and the filter further reduces question-to-bug errors at the cost of bug recall.

The two approaches make different errors. Fine-tuning finds more bugs but over-predicts the bug label, whereas filtered RAG makes fewer false bug predictions at most model sizes,

and the question-to-bug confusion persists under every method we test. RAG's inference is slower, and at Qwen-32B its total time exceeds fine-tuning's even when training is included, while fine-tuning must be repeated whenever labels are added or the model is replaced.

Since our study covers one model family and one balanced benchmark, future work can focus on the generalizability of these findings by extending them to other LLM families and quantization settings to further strengthen or bound the conclusions drawn here. Additional directions include examining imbalanced label distributions, [test these findings on other LLM families and quantization settings, on imbalanced, minority-class scenarios, and multi-label classification, and other issue tracking platforms data, and on other issue trackers](#) such as Jira and Bugzilla. Further research should also explore stronger retrieval-side interventions, such as LLM-based filtering or re-ranking of retrieved neighbors, as well as the effects of different embedding models and similarity metrics on retrieval quality. [Since the two approaches make different errors, combining retrieval with a fine-tuned model may combine their strengths. Many remaining errors are questions filed under a bug template, which we hypothesize draws bug-labeled neighbors, so re-ranking the neighbors with an LLM, other embedding models, and other similarity metrics may retrieve better examples. Finally, future studies should investigate alternative other prompt structures, including advanced strategies such as Chain-of-Thought such as chain-of-thought prompting \[52\], may help the LLM weigh an issue's content over its template.](#)

DATA AVAILABILITY

Our replication package [53] contains the code, data, and all necessary materials to reproduce the results presented in this paper.

REFERENCES

- [1] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, "Leveraging GPT-like LLMs to automate issue labeling," in *Proceedings of the 21st International Conference on Mining Software Repositories*, 2024, pp. 469–480.
- [2] Z. Liao, D. He, Z. Chen, X. Fan, Y. Zhang, and S. Liu, "Exploring the characteristics of issue-related behaviors in GitHub using visualization techniques," *IEEE Access*, vol. 6, pp. 24003–24015, 2018.
- [3] G. Catolino, F. Palomba, A. Zaidman, and F. Ferrucci, "Not all bugs are the same: Understanding, characterizing, and classifying bug types," *Journal of Systems and Software*, vol. 152, pp. 165–181, 2019.
- [4] H. Wu, Y. Ma, Z. Xiang, C. Yang, and K. He, "A spatial-temporal graph neural network framework for automated software bug triaging," *Knowledge-Based Systems*, vol. 241, p. 108308, 2022.
- [5] F. Santos, J. Vargovich, B. Trinkenreich, I. Santos, J. Penney, R. Britto, J. F. Pimentel, I. Wiese, I. Steinmacher, A. Sarma *et al.*, "Tag that issue: applying API-domain labels in issue tracking systems," *Empirical Software Engineering*, vol. 28, no. 5, p. 116, 2023.
- [6] G. Aracena, K. Luster, F. Santos, I. Steinmacher, and M. A. Gerosa, "Applying large language models to issue classification: Revisiting with extended data and new models," *Science of Computer Programming*, vol. 246, p. 103333, 2025.
- [7] R. Kallis, A. Di Sorbo, G. Canfora, and S. Panichella, "Ticket tagger: Machine learning driven issue classification," in *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2019, pp. 406–409.
- [8] G. Antoniol, K. Ayari, M. Di Penta, F. Khomh, and Y.-G. Guéhéneuc, "Is it a bug or an enhancement? a text-based approach to classify change requests," in *Proceedings of the 2008 conference of the center for advanced studies on collaborative research: meeting of minds*, 2008, pp. 304–318.
- [9] K. Herzig, S. Just, and A. Zeller, "It's not a bug, it's a feature: how misclassification impacts bug prediction," in *2013 35th international conference on software engineering (ICSE)*. IEEE, 2013, pp. 392–401.
- [10] Q. Fan, Y. Yu, G. Yin, T. Wang, and H. Wang, "Where is the road for issue reports classification based on text mining?" in *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 2017, pp. 121–130.
- [11] M. Izadi, "CatIss: An intelligent tool for categorizing issues reports using transformers," in *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, 2022, pp. 44–47.
- [12] R. Kallis, G. Colavito, A. Al-Kaswan, L. Pascarella, O. Chaparro, and P. Rani, "The NLBSE'24 tool competition," in *Proceedings of the Third ACM/IEEE International Workshop on NL-based Software Engineering*, 2024, pp. 33–40.
- [13] G. Colavito, F. Lanubile, and N. Novielli, "Benchmarking large language models for automated labeling: The case of issue report classification," *Information and Software Technology*, vol. 184, p. 107758, 2025.
- [14] J. Heo and S. Lee, "A study on applying large language models to issue classification," in *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*. IEEE Computer Society, 2025, pp. 136–146.
- [15] N. Pandey, D. K. Sanyal, A. Hudait, and A. Sen, "Automated classification of software issue reports using machine learning techniques: an empirical study," *Innovations in Systems and Software Engineering*, vol. 13, no. 4, pp. 279–297, 2017.
- [16] A. Trautsch and S. Herbold, "Predicting issue types with seBERT," in *Proceedings of the 1st international workshop on natural language-based software engineering*, 2022, pp. 37–39.
- [17] G. Colavito, F. Lanubile, and N. Novielli, "Issue report classification using pre-trained language models," in *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, 2022, pp. 29–32.
- [18] J. Wang, X. Zhang, and L. Chen, "How well do pre-trained contextual language representations recommend labels for GitHub issues?" *Knowledge-Based Systems*, vol. 232, p. 107476, 2021.
- [19] G. Aracena, K. Luster, F. Santos, I. Steinmacher, and M. A. Gerosa, "Applying large language models to issue classification," in *Proceedings of the Third ACM/IEEE International Workshop on NL-based Software Engineering*, 2024, pp. 57–60.
- [20] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.
- [21] T. Brown, B. Mann, N. Ryder *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [22] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, and W. Chen, "What makes good in-context examples for GPT-3?" in *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd workshop on knowledge extraction and integration for deep learning architectures*, 2022, pp. 100–114.
- [23] G. Yu, L. Liu, H. Jiang, S. Shi, and X. Ao, "Retrieval-augmented few-shot text classification," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 6721–6735.
- [24] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.
- [25] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, "Impact of data quality for automatic issue classification using pre-trained language models," *Journal of Systems and Software*, vol. 210, p. 111838, 2024.
- [26] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, 2019, pp. 3982–3992.
- [27] H. Huang, R. Widayarsi, T. Zhang, I. C. Irsan, J. Shi, H. W. Ang, F. Liauw, E. L. Ouh, L. K. Shar, H. J. Kang *et al.*, "Back to the basics: Rethinking issue-commit linking with LLM-assisted retrieval," *arXiv preprint arXiv:2507.09199*, 2025.
- [28] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The FAISS library," *IEEE Transactions on Big Data*, 2025.
- [29] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE transactions on information theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [30] S. A. Dudani, "The distance-weighted k-nearest-neighbor rule," *IEEE Transactions on Systems, Man, and Cybernetics*, no. 4, pp. 325–327, 1976.
- [31] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the association for computational linguistics*, vol. 12, pp. 157–173, 2024.
- [32] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," *Advances in neural information processing systems*, vol. 36, pp. 10088–10115, 2023.
- [33] GitHub, "GitHub Copilot," <https://github.com/features/copilot>, 2025.
- [34] GitLab, "GitLab Duo," <https://about.gitlab.com/gitlab-duo/>, 2025.
- [35] L. Khatib, N. S. Mathews, and M. Nagappan, "AssertFlip: Reproducing bugs via inversion of LLM-generated passing tests," *arXiv preprint arXiv:2507.17542*, 2025.
- [36] X. Wang, P. Gao, X. Meng, C. Peng, R. Hu, Y. Lin, and C. Gao, "Aegis: An agent-based framework for general bug reproduction from issue descriptions," *arXiv preprint arXiv:2411.18015*, 2024.
- [37] T. Ahmed, J. Ganhotra, R. Pan, A. Shinnar, S. Sinha, and M. Hirzel, "Otter: Generating tests from issues to validate swe patches," *arXiv preprint arXiv:2502.05368*, 2025.
- [38] E. Dinç and E. Tüzün, "Judge the votes: A system to classify bug reports and give suggestions," in *2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware)*. IEEE, 2025, pp. 01–10.
- [39] S. Bharadwaj and T. Kadam, "GitHub issue classification using BERT-style models," in *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, 2022, pp. 40–43.
- [40] M. Izadi, K. Akbari, and A. Heydarnoori, "Predicting the objective and priority of issue reports in software repositories," *Empirical Software Engineering*, vol. 27, no. 2, p. 50, 2022.
- [41] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.
- [42] J. Von der Mosel, A. Trautsch, and S. Herbold, "On the validity of pre-trained transformers for natural language processing in the software engineering domain," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1487–1507, 2022.
- [43] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, "Large language models for issue report classification," in *Ital-IA 2024, ser. CEUR Workshop Proceedings*, vol. 3762, 2024, pp. 18–23.
- [44] M. L. Siddiq and J. C. Santos, "BERT-based GitHub issue report classification," in *Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering*, 2022, pp. 33–36.

- [45] G. Colavito, F. Lanubile, and N. Novielli, “Few-shot learning for issue report classification,” in *2023 IEEE/ACM 2nd International Workshop on Natural Language-Based Software Engineering (NLBSE)*. IEEE, 2023, pp. 16–19.
- [46] M. Assi, S. Hassan, and Y. Zou, “LLM-Cure: LLM-based competitor user review analysis for feature enhancement,” *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 4, pp. 1–25, 2026.
- [47] V.-H. Le and H. Zhang, “Log parsing with prompt-based few-shot learning,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2438–2449.
- [48] H. Ma, C. Zhang, Y. Bian, L. Liu, Z. Zhang, P. Zhao, S. Zhang, H. Fu, Q. Hu, and B. Wu, “Fairness-guided few-shot prompting for large language models,” in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023, pp. 43 136–43 155.
- [49] R. Logan IV, I. Balazevic, E. Wallace, F. Petroni, S. Singh, and S. Riedel, “Cutting down on prompts and parameters: Simple few-shot learning with language models,” in *Findings of the Association for Computational Linguistics: ACL 2022*. Dublin, Ireland: Association for Computational Linguistics, 2022, pp. 2824–2835.
- [50] A. Milios, S. Reddy, and D. Bahdanau, “In-context learning for text classification with many labels,” in *Proceedings of the 1st GenBench Workshop on (Benchmarking) Generalisation in NLP*. Singapore: Association for Computational Linguistics, 2023, pp. 173–184.
- [51] A. Akhavan, A. Hoseinpour, A. Heydarnoori, H. Bagheri, and M. Keshani, “LinkAnchor: An autonomous LLM-based agent for issue-to-commit link recovery,” in *To Appear in Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*, July 2026, preprint available at <https://arxiv.org/abs/2508.12232>.
- [52] A. Koyuncu, “Exploring fine-grained bug report categorization with large language models and prompt engineering: An empirical study,” *ACM Transactions on Software Engineering and Methodology*, 2025.
- [53] “Replication package: Can retrieval-augmented few-shot prompting match LoRA fine-tuning for issue report classification? an empirical study,” <https://figshare.com/s/2c7a6db860c611246774>, Accessed: 05-18-2026.